

Industry Solutions Framework

Cortex Code Industry Skills
Development Life Cycle

Health Sciences
Industry Solutions Tiger Team

Built with Cortex Code | March 2026

Table of Contents

1. Introduction
2. Architecture Overview
3. Skill Development Lifecycle
4. Lifecycle Phases
5. Skill Taxonomy & Naming Convention
6. Healthcare Skills Inventory
7. Cortex Knowledge Extensions & Data Model Knowledge
8. Snowflake Platform Skills Integration
9. Cross-Domain Solution Patterns
10. Functional Validation: Clinical Trial Protocol Design
11. Getting Started
12. Key Design Considerations

1. Introduction

The Industry Solutions Framework defines a modular, skill-based architecture for building healthcare and life sciences data solutions on Snowflake using Cortex Code. It provides a structured approach to developing, hardening, and distributing industry-specific AI coding skills through a two-repository model that separates rapid experimentation from production-grade delivery.

This document consolidates the Healthcare Solutions architecture guide and the Industry Solutions Framework lifecycle into a single reference. All content reflects the current design as of March 2026, including the flat skills/ directory structure, hcsl-* naming convention, health-sciences-incubator profile, systemPromptRepo-based orchestrator loading, and GitHub-based remote skill discovery.

What This Framework Delivers

- **17 Health Sciences Skills:** 15 domain skills + 2 Cortex Knowledge Extension (CKE) skills, all registered as REMOTE from GitHub
- **1 Orchestrator Agent:** health-sciences-incubator.md with intent detection, domain routing, CKE integration, 9 cross-domain patterns, and HIPAA guardrails
- **1 Profile:** health-sciences-incubator with systemPromptRepo pointing to the orchestrator agent file on GitHub
- **Flat Skill Discovery:** All skills at skills/{hcsl-name}/SKILL.md (1 level from scan root) for Cortex Code auto-discovery
- **Platform Integration:** Skills invoke 25+ bundled Snowflake platform skills (Dynamic Tables, Cortex AI, Streamlit, SPCS, dbt, ML, governance)
- **Knowledge Grounding:** CKEs for PubMed/ClinicalTrials.gov + Cortex Search over DICOM data model reference

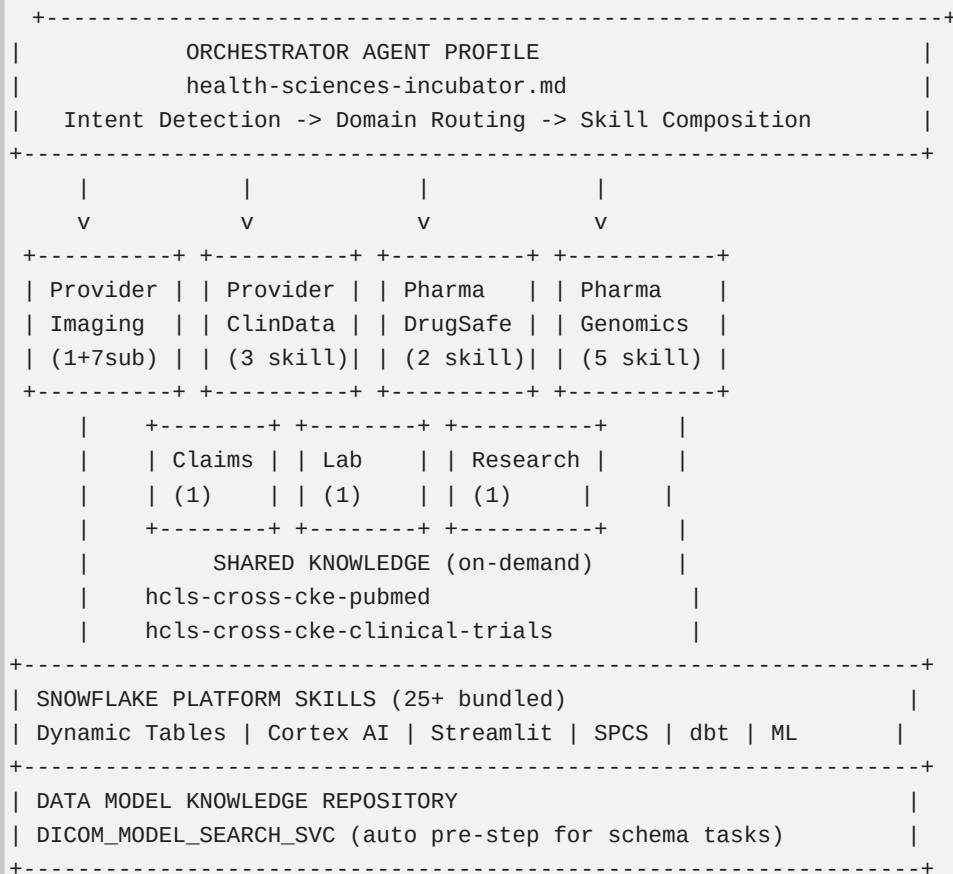
Key Benefits

- **Modularity:** Reusable skills that encode domain expertise and best practices
- **Intelligent Routing:** Orchestrator routes to the right skill based on natural language intent
- **Composability:** Cross-domain patterns compose multiple skills for complex use cases
- **Governance by Default:** HIPAA guardrails enforced as cross-cutting concerns
- **Single-Command Onboarding:** cortex profile add health-sciences-incubator delivers everything

2. Architecture Overview

The architecture follows a layered model: an orchestrator agent profile at the top, healthcare domain skill categories in the middle (with shared CKE skills invoked on-demand), Snowflake platform skills at the foundation, and data model knowledge repositories feeding schema context upward.

Layered Architecture



How It Works

Step	Action	Component
1	User sends natural language request	Cortex Code CLI
2	Orchestrator detects healthcare domain	health-sciences-incubator.md
3	Matching domain skill invoked automatically	SKILL.md routing
4	Schema tasks auto-query Data Model Knowledge	DICOM_MODEL_SEARCH_SVC
5	Domain skills invoke CKEs for evidence	CKE Cortex Search Services
6	Platform skills invoked for infrastructure	Dynamic Tables, Streamlit, etc.

7	HIPAA guardrails applied across workflows	Masking, RLS, Audit
---	---	---------------------

3. Skill Development Lifecycle

The framework establishes a two-repo model separating rapid experimentation from production-grade skill delivery, with a Tiger Team serving as the quality gateway.

Two-Repo Model

The framework uses two distinct repositories to separate experimentation from production:

	Incubator Repo	SFS Skills Repo
GitHub Org/Repo	Snowflake-Solutions/ health-sciences-incubator (industry-specific repo)	Snowflake-Solutions/ cortex-code-skills (shared SFS skills repo)
Used in Phases	Phase 0 (setup) and Phase 1 (incubate)	Phase 2 (harden and promote to production)
Purpose	Rapid skill development and experimentation	Approved, production- grade skills
Who contributes	Anyone (SEs, SAs, field)	Tiger Team only
Branch/PR rules	Open branching, lightweight review	Formal PR, CI gates, promotion stages
Quality bar	Low - iterate fast	High - best practices, evidence, testing
Skills flow	Skills created and refined here	Mature skills graduate here from incubator

Two-Profile Model

	Incubator Profile	Production Profile
Name	health-sciences-incubator	health-sciences-solutions
Purpose	Test experimental skills	Deliver approved skills
Skills source	Incubator repo (all skills)	SFS repo (production only)
Who uses	Contributors and testers	Field teams
Command	cortex profile add health-sciences-incubator	cortex profile add health-sciences-solutions

Profile Configuration (Current)

The incubator profile is configured at `~/.snowflake/cortex/profiles/health-sciences-incubator.json` with three critical fields:

- **skillRepos:** Points to the GitHub repo skills/ subdirectory for remote skill discovery

- **systemPromptRepo:** Points to agents/health-sciences-incubator.md on GitHub (NOT a systemPrompt string)
- **Skill discovery:** Cortex Code scans skills/{name}/SKILL.md at exactly 1 level depth from the scan root

```
{
  "name": "health-sciences-incubator",
  "skillRepos": [{
    "ref": "main",
    "source": "https://github.com/<owner>/coco-healthcare-skills/tree/main/skills",
    "url": "https://github.com/<owner>/coco-healthcare-skills.git"
  }],
  "systemPromptRepo": {
    "ref": "main",
    "source": ".../tree/main/agents/health-sciences-incubator.md",
    "url": "https://github.com/<owner>/coco-healthcare-skills.git"
  }
}
```

4. Lifecycle Phases

Phase 0: Framework Setup (Tiger Team)

Phase 0 establishes the incubator repository in the Snowflake-Solutions GitHub organization. This is an industry-specific repo (e.g., Snowflake-Solutions/health-sciences-incubator) separate from the shared SFS cortex-code-skills repo. All Phase 0 and Phase 1 work happens here.

```
GitHub Org:  Snowflake-Solutions
Repo:        health-sciences-incubator  (industry-specific)
Access:      Open to all SEs, SAs, and field team members
```

- Create the industry-specific incubator repo in Snowflake-Solutions GitHub org
- Define skill development guidelines, templates, and quality bar for promotion
- Set up flat skills/ directory structure with hcls-* naming convention
- Create incubator profile (health-sciences-incubator) with systemPromptRepo pointing to this repo
- Create orchestrator agent .md with routing rules, taxonomy tree, guardrails
- Document full lifecycle: incubator repo -> SFS cortex-code-skills repo -> profile -> field

Phase 1: Incubate & Iterate (Contributors)

All Phase 1 work happens in the same industry-specific incubator repo (Snowflake-Solutions/health-sciences-incubator). Any SE, SA, or field team member can contribute. This is a sandbox for rapid iteration with no formal approval gates.

```
GitHub Org:  Snowflake-Solutions
Repo:        health-sciences-incubator  (same repo as Phase 0)
Access:      Open branching, no formal PR approval needed
```

- Branch freely in the incubator repo, create and refine skills collaboratively
- Create skills following hcls-{sub}-{func}-{skill} naming convention
- Test via: cortex profile add health-sciences-incubator
- Or add skills directly: cortex skill add <incubator-github-url>/tree/main/skills
- Use cortex skill update to refresh skills from GitHub after repo changes
- When skill is mature, signal to Tiger Team for Phase 2 promotion

Phase 2: Harden & Promote (Tiger Team Only)

Phase 2 is where skills graduate from the incubator repo to the shared SFS skills repo. Tiger Team picks up mature skills from Snowflake-Solutions/health-sciences-incubator and promotes them into Snowflake-Solutions/cortex-code-skills -- the production-grade SFS repo that serves all Cortex Code industry skills across Snowflake.

FROM: Snowflake-Solutions/health-sciences-incubator (industry incubator)
 TO: Snowflake-Solutions/cortex-code-skills (SFS production repo)
 WHO: Tiger Team only -- no direct field contributions to SFS repo

1. Evaluate & Accept

Review skill from incubator for relevance and quality potential. Decide: accept, request changes, or decline.

2. Audit & Refine

Audit against SKILL_BEST_PRACTICES.md. Validate no overlap with bundled product skills. Refine workflows, stopping points, guardrails.

3. Test & Validate

Test with realistic customer scenarios. Gather testers (3+ for staging, 5+ for production). Record evidence in skill_evidence.yaml.

4. Submit & Promote

Submit PR to SFS repo at draft stage. Promote through: draft -> review -> staging -> production.

5. Integrate into Production Profile

Add approved skill to health-sciences-solutions profile. Update orchestrator routing. Wire CKEs and data model knowledge.

Phase 3: Publish & Distribute (Tiger Team)

- Production profile references only production-stage skills
- Publish to Snowflake registry: cortex profile publish health-sciences-solutions
- Document prerequisites (Marketplace CKEs, Cortex Search setup)

Phase 4: Consume & Feedback (Field Teams)

- cortex profile add health-sciences-solutions for production skills
- cortex profile add health-sciences-incubator to preview upcoming skills
- File issues/feedback -> feeds back into Phase 1 incubator
- cortex profile sync for updates

5. Skill Taxonomy & Naming Convention

Skills follow a five-level hierarchy: Industry > Sub-Industry > Business Function > Use Case Skill > Sub-Skill. The hierarchy is encoded in the skill name using a flat naming convention rather than deep directory nesting.

Naming Pattern

```
hcls-{sub-industry}-{function}-{skill}
```

Component	Values	Notes
hcls	Fixed prefix	Health Sciences industry identifier
sub-industry	provider, pharma, payer, cross	Maps to sub-industry taxonomy level
function	imaging, cdata, dsafety, genomics, lab, claims, research, cke	Short alias for business function
skill	dicom-parser, pharmacovigilance, nextflow, etc.	Use case skill name (leaf level)

Repository Directory Structure (Current)

The incubator repo uses a FLAT skills/ directory. Cortex Code requires exactly 1 level of nesting from the scan root: skills/{name}/SKILL.md. The taxonomy hierarchy is encoded in the hcls-* name prefix, not in directory depth.

```
skills/
hcls-provider-imaging/           # Router + 7 sub-skills inside
hcls-provider-imaging-dicom-parser/  # Standalone DICOM parser
hcls-provider-cdata-fhir/
hcls-provider-cdata-clinical-nlp/
hcls-provider-cdata-omop/
hcls-provider-claims-data-analysis/
hcls-pharma-dsafety-pharmacovigilance/
hcls-pharma-dsafety-clinical-trial-protocol/
hcls-pharma-genomics-nextflow/
hcls-pharma-genomics-variant-annotation/
hcls-pharma-genomics-single-cell-qc/
hcls-pharma-genomics-scvi-tools/
hcls-pharma-genomics-survival-analysis/
hcls-pharma-lab-allotrope/
hcls-cross-research-problem-selection/
hcls-cross-cke-pubmed/
hcls-cross-cke-clinical-trials/
```

Sub-Industries

Sub-Industry	Customer Types	Business Functions
Provider	Hospitals, health systems, clinics	Clinical Research, Clinical Data Mgmt, Revenue Cycle
Pharma	Pharma, biotech, CROs	Drug Safety, Genomics, Lab Operations
Payer	Health plans, TPAs, PBMs	Claims Processing (future)
Cross-Industry	All of the above	Research Strategy, Knowledge Extensions

6. Healthcare Skills Inventory

17 skills organized by sub-industry and business function. All registered as REMOTE from GitHub.

6.1 Provider > Clinical Research (Imaging)

hcls-provider-imaging is a router skill with 7 sub-skills. hcls-provider-imaging-dicom-parser is a standalone skill.

Skill / Sub-Skill	Triggers	Description
hcls-provider-imaging (router)	DICOM, medical imaging, radiology, imaging pipeline	Routes to 7 sub-skills covering full imaging lifecycle
dicom-parser (sub)	DICOM parse, extract tags	18-table data model + pydicom parser
dicom-ingestion (sub)	Ingest DICOM, load images	Stages, COPY, Dynamic Tables
dicom-analytics (sub)	Imaging analytics, NLP	Cortex AI on reports, metrics
imaging-viewer (sub)	DICOM viewer, Streamlit	Streamlit dashboard + SPCS viewer
imaging-governance	HIPAA, PHI masking	Masking policies, audit trails
imaging-ml (sub)	Radiology AI, pathology	ML training, Model Registry
data-model-knowledge	Schema lookup, PHI cols	Cortex Search auto pre-step
hcls-provider-imaging-dicom-parser (standalone)	DICOM data model, pydicom	Standalone DICOM parser skill

6.2 Provider > Clinical Data Management

Skill	Triggers	Description
hcls-provider-cdata-fhir	FHIR, HL7, Patient, Observation, Bundle	FHIR R4 resources to relational tables
hcls-provider-cdata-clinical-nlp	Clinical NLP, NER, clinical notes, ICD	Structured extraction from clinical text
hcls-provider-cdata-omop	OMOP, CDM, OHDSI, vocabulary mapping	EHR/claims to OMOP CDM v5.4

6.3 Provider > Revenue Cycle

Skill	Triggers	Description
hcls-provider-claims-data-analysis	Claims, RWE, 837/835, utilization, HEDIS	Cohort building, utilization metrics, treatment patterns

6.4 Pharma > Drug Safety

Skill	Triggers	Description
hcls-pharma-dsafety-pharmacovigilance	FAERS, adverse events, drug safety, MedDRA	FDA FAERS signal detection
hcls-pharma-dsafety-clinical-trial-protocol	Clinical trial protocol, FDA submission	Protocol generation via waypoint architecture

6.5 Pharma > Genomics

Skill	Triggers	Description
hcls-pharma-genomics-nextflow	nf-core, Nextflow, FASTQ	Runs nf-core pipelines (rnaseq, sarek)
hcls-pharma-genomics-variant-annotation	VCF, ClinVar, gnomAD	Variant pathogenicity annotation
hcls-pharma-genomics-single-cell-qc	scRNA-seq, QC, scanpy	MAD-based QC using scverse
hcls-pharma-genomics-scvi-tools	scVI, scANVI, totalVI	Deep learning single-cell analysis
hcls-pharma-genomics-survival-analysis	Kaplan-Meier, Cox, PFS	Time-to-event analysis

6.6 Pharma > Lab Operations & Cross-Industry

Skill	Triggers	Description
hcls-pharma-lab-allotrope	Instrument files, ASM, LIMS, ELN	Lab instrument to Allotrope JSON/CSV
hcls-cross-research-problem-selection	Research problem, project ideation	Systematic problem selection
hcls-cross-cke-pubmed	PubMed, biomedical literature	RAG search over PubMed corpus
hcls-cross-cke-clinical-trials	ClinicalTrials.gov, trial search	RAG search over clinical trials

7. Cortex Knowledge Extensions & Data Model Knowledge

CKEs as Composable Skills

CKEs are Snowflake Marketplace shared Cortex Search Services packaged as standalone composable skills. Domain skills invoke them on-demand when evidence grounding adds value. This eliminates duplication and makes adding new CKEs trivial.

CKE Skill	Data Source	Marketplace ID
hcls-cross-cke-pubmed	PubMed biomedical literature	GZSTZ67BY9OQW
hcls-cross-cke-clinical-trials	ClinicalTrials.gov registry	GZSTZ67BY9ORD

CKE Routing

CKE Skill	Domain Skills That Invoke It
hcls-cross-cke-pubmed	pharmacovigilance, clinical-nlp, research-problem-selection, dicom-analytics
hcls-cross-cke-clinical-trials	clinical-trial-protocol, claims-data-analysis, survival-analysis

Data Model Knowledge Repository

Reference data models are stored as searchable knowledge on Snowflake using Cortex Search. The DICOM model covers 18 tables and 222 columns. Skills query the repository at runtime for live schema context instead of hardcoded definitions.

```
Excel -> CSV -> Snowflake Table -> Cortex Search Service
DICOM_MODEL_SEARCH_SVC <-- Skills query at runtime (auto pre-step)
```

Key design decision: Data Model Knowledge auto-fires (Step 0) because schema context is always needed for DICOM tasks. CKEs are on-demand -- the domain skill decides when literature/trial evidence adds value.

8. Snowflake Platform Skills Integration

Healthcare domain skills leverage 25+ bundled Snowflake platform skills for infrastructure, compute, AI, governance, and visualization.

Data Engineering

- **Dynamic Tables:** Incremental pipelines for DICOM metadata, FHIR resources, claims aggregation
- **Streams & Tasks:** Event-driven processing of new imaging/HL7 data
- **dbt Projects:** OMOP CDM vocabulary mapping, claims analytics, HEDIS measures

AI & Machine Learning

- **Cortex AI Functions:** EXTRACT_ANSWER, SUMMARIZE, COMPLETE, SENTIMENT on clinical data
- **Cortex Search:** Imaging metadata search, data model knowledge, CKE integration
- **ML Registry:** Imaging classifiers, survival models registered for SQL inference

Applications & Visualization

- **Streamlit in Snowflake:** Imaging dashboards, pharmacovigilance signals, clinical explorers
- **SPCS:** DICOM pixel viewer (OHIF/Cornerstone.js), deep learning inference, bioinformatics

Security & Governance

- **Sensitive Data Classification:** SYSTEM\$CLASSIFY for PHI auto-detection
- **Data Masking:** IS_ROLE_IN_SESSION() based masking for patient names, IDs, dates
- **Row-Access Policies:** Institutional data segregation by role
- **Audit Trails:** ACCESS_HISTORY monitoring for all PHI-containing tables

9. Cross-Domain Solution Patterns

The most powerful capability of the orchestrator is composing multiple skills for solutions that span healthcare business domains. The user simply describes their business need in natural language, and the orchestrator automatically selects and chains the appropriate skills. Below are 9 pre-defined composition patterns.

Pattern 1: Imaging + Clinical Integration

- Step 1: Build imaging metadata tables
- Step 2: Ingest FHIR DiagnosticReport
- Step 3: Extract findings from radiology reports
- Step 4: Enrich with PubMed research
- Step 5: Build Streamlit dashboard

Pattern 2: Clinical Data Warehouse (OMOP)

- Step 1: Ingest FHIR bundles
- Step 2: Transform to OMOP CDM with vocabulary mapping
- Step 3: Apply HIPAA governance
- Step 4: Create semantic views for analytics
- Step 5: Build clinical dashboards

Pattern 3: Drug Safety Signal Detection

- Step 1: Load and analyze FAERS data
- Step 2: Search PubMed for drug-event associations
- Step 3: Extract adverse events from narrative text
- Step 4: Correlate with claims utilization
- Step 5: Build safety signal dashboard

Pattern 4: Genomics + Clinical Outcomes

- Step 1: Run nf-core pipeline on sequencing data
- Step 2: Annotate variants with ClinVar/gnomAD
- Step 3: Correlate variants with patient outcomes
- Step 4: Train predictive models

Pattern 5: Single-Cell Analysis Pipeline

- Step 1: QC and filter scRNA-seq data
- Step 2: Deep learning integration and batch correction
- Step 3: Register models in Snowflake ML Registry

Pattern 6: Real-World Evidence Study

- Step 1: Build cohorts from claims data
- Step 2: Cross-reference with registered trials
- Step 3: Standardize to OMOP CDM
- Step 4: Time-to-event outcomes analysis
- Step 5: Validate against published literature

Step 6: Study results dashboard

Pattern 7: Clinical Trial Design

Step 1: Validate research problem

Step 2: Search for similar/competing trials

Step 3: Review biomedical literature

Step 4: Generate protocol document

Step 5: Power analysis and endpoint design

Step 6: Feasibility from claims data

Pattern 8: Lab Data Modernization

Step 1: Standardize instrument outputs to Allotrope

Step 2: Incremental pipeline via Dynamic Tables

Step 3: Lab analytics dashboard

Pattern 9: Clinical Data Application (React)

Step 1: Prepare backend data (FHIR, OMOP, imaging, claims)

Step 2: Build React/Next.js app with Snowflake data

Step 3: Deploy to SPCS

Step 4: Enforce PHI masking at API layer

10. Functional Validation: Clinical Trial Protocol Design

This walkthrough demonstrates how the orchestrator routes a clinical trial design request through multiple skills (Pattern 7), validating the full chain from profile to orchestrator to skills to platform integration.

Scenario

A user types the following natural language prompt into Cortex Code:

```
"Design a Phase III clinical trial for a novel GLP-1 receptor agonist targeting Type 2 Diabetes with cardiovascular outcome endpoints."
```

The user does not need to know which skills exist. The orchestrator automatically detects keywords ('clinical trial', 'Phase III', 'GLP-1', 'Type 2 Diabetes') and identifies this as a Clinical Trial Design pattern (Pattern 7), then plans and executes a multi-skill chain.

Execution Chain (Automatic)

1. Research Problem Validation

Evaluates the research hypothesis using Fischbach & Walsh decision trees. Validates scientific merit, feasibility, and novelty.

2. Competitor Trial Search

Searches ClinicalTrials.gov for existing GLP-1 trials with cardiovascular endpoints. Returns competing/similar trials, enrollment status, and design patterns.

3. Literature Evidence Review

Searches PubMed for GLP-1 cardiovascular outcome evidence. Grounds the protocol in published clinical evidence.

4. Protocol Generation

Generates the full protocol document: study design, inclusion/exclusion criteria, primary/secondary endpoints, statistical plan, safety monitoring.

5. Endpoint Design & Power Analysis

Designs time-to-event endpoints (MACE), calculates sample size requirements, defines statistical methods.

6. Feasibility Analysis

Analyzes claims data for T2D patient population size, geographic distribution, and comorbidity profiles to validate enrollment feasibility.

QA Validation Results

This pattern was validated end-to-end. All 4 validation phases passed:

- Phase 1: Profile -> systemPromptRepo -> orchestrator agent file (PASS)
- Phase 2: Orchestrator skill references match registered skills (PASS)
- Phase 3: SKILL.md frontmatter (name, description, parent_skill) consistency (PASS)

- Phase 4: Sub-skill parent_skill references and routing table consistency (PASS)
- Functional: Clinical trial protocol design dry-run plan generated successfully (PASS)

11. Getting Started

This section walks through the complete setup process: connecting to the incubator GitHub repository, adding the profile and skills to your Cortex Code environment, validating the setup, and using the framework for healthcare use cases.

Prerequisites

- Cortex Code CLI installed and authenticated to your Snowflake account
- GitHub access to the coco-healthcare-skills repository
- A Snowflake account with a warehouse for query execution
- (Optional) Snowflake Marketplace access for CKE listings (PubMed, Clinical Trials)

Step 1: Connect to the Incubator GitHub Repository

The incubator repository hosts all 17 health sciences skills, the orchestrator agent file, reference data models, and setup scripts. Start by cloning the repo locally:

```
git clone https://github.com/<owner>/coco-healthcare-skills.git
cd coco-healthcare-skills
```

The repository structure is:

```
coco-healthcare-skills/
agents/                # Orchestrator agent .md files
  health-sciences-incubator.md
skills/                # 17 flat skill directories (hcls-*)
  hcls-provider-imaging/
  hcls-provider-cdata-fhir/
  hcls-pharma-dsafety-pharmacovigilance/
  ...                  # (17 total)
references/            # Data model spreadsheets
scripts/              # Setup and utility scripts
```

Step 2: Add the Profile to Cortex Code

The fastest way to get all skills and the orchestrator loaded is to add the incubator profile. This single command registers all 17 skills as REMOTE from GitHub and loads the orchestrator agent via systemPromptRepo:

```
cortex profile add health-sciences-incubator
```

Alternatively, if the profile is not yet published to the Snowflake registry, you can add skills directly from the GitHub repository URL:

```
# Add all skills from the GitHub repo skills/ directory
cortex skill add "https://github.com/<owner>/coco-healthcare-skills/tree/main/skills"
```

Step 3: Validate Your Setup

After adding the profile, verify that everything loaded correctly:

```
# Check that the profile is active
cortex profile show health-sciences-incubator

# Verify all 17 skills are registered
/skill
# Expected: 17 hcls-* skills listed as REMOTE

# Verify the orchestrator agent is loaded
/agents
# Expected: health-sciences-incubator listed
```

If skills are not showing, refresh them from the GitHub source:

```
cortex skill update
```

Step 4: Set Up Optional Dependencies

For full functionality, set up these optional Snowflake-side dependencies:

Cortex Knowledge Extensions (CKEs)

Install PubMed and/or Clinical Trials CKE from Snowflake Marketplace. These provide RAG-based literature search that domain skills invoke on-demand for evidence grounding. Skills work without CKEs but provide richer results with them.

Data Model Knowledge Repository

Run the provided setup script to create the Cortex Search Service over the DICOM data model:

```
scripts/setup_dicom_model_knowledge_repo.sql
```

This enables the auto pre-step pattern where imaging skills query live schema definitions before generating DDL, COPY INTO mappings, or governance policies.

Step 5: Start Using the Framework

Once setup is complete, simply ask healthcare-related questions in natural language. The orchestrator automatically detects the domain, selects the right skills, and composes them into a solution. You do NOT need to know skill names -- just describe your business need.

Example Prompts

Below are example prompts organized by business domain. The orchestrator handles routing automatically.

Medical Imaging

```
"I have DICOM files from our radiology department on S3. Build a complete pipeline to parse, ingest, and analyze the imaging metadata on Snowflake."
```

Clinical Data / EHR

"We receive FHIR R4 bundles from our EHR. Transform Patient, Condition, and Observation resources into analytics-ready tables on Snowflake."

Drug Safety

"Analyze FDA FAERS data for adverse events associated with aspirin. Detect safety signals and cross-reference with published literature."

Clinical Trials

"Design a Phase III clinical trial for a novel GLP-1 receptor agonist targeting Type 2 Diabetes with cardiovascular outcome endpoints."

Genomics

"I have whole-genome sequencing FASTQ files. Run variant calling and annotate pathogenic variants with ClinVar and gnomAD frequencies."

Claims / RWE

"Build a retrospective cohort of Type 2 Diabetes patients from our claims data and analyze treatment patterns and medication adherence."

Lab Data

"We have instrument output files from our HPLC analyzers. Convert them to Allotrope Simple Model format for our LIMS integration."

Research Strategy

"I have an idea to study the correlation between gut microbiome and immunotherapy response. Help me evaluate if this is a good project."

Cross-Domain (Multi-Skill)

"Build a real-world evidence study: cohort from claims, standardize to OMOP, run survival analysis, and validate against published literature."

Step 6: Keep Skills Updated

As the incubator evolves, pull the latest skills from GitHub:

```
# Refresh all skills from GitHub
cortex skill update

# Or sync the entire profile
cortex profile sync health-sciences-incubator
```

Key File Locations

Component	Path
Profile	~/snowflake/cortex/profiles/ health-sciences-incubator.json
Orchestrator Agent	agents/health-sciences-incubator.md (loaded via systemPromptRepo from GitHub)
Skills (repo root)	skills/ (17 hcls-* directories)
Reference Model	references/dicom_data_model_reference.xlsx
Setup SQL	scripts/setup_dicom_model_knowledge_repo.sql

Snowflake Objects

Object	Fully Qualified Name
Data Model Table	UNSTRUCTURED_HEALTHDATA.DATA_MODEL_KNOWLEDGE. DICOM_MODEL_REFERENCE
Cortex Search Svc	UNSTRUCTURED_HEALTHDATA.DATA_MODEL_KNOWLEDGE. DICOM_MODEL_SEARCH_SVC
Stage	UNSTRUCTURED_HEALTHDATA.DATA_MODEL_KNOWLEDGE. dicom_model_stage

12. Key Design Considerations

Ten key design decisions that shaped the framework:

Skill Discovery Constraint

Cortex Code scanner expects skills at exactly 1 level depth from scan root: skills/{name}/SKILL.md. Deep nesting breaks discovery. Solution: flat skills/ dir with hcls-* prefix encoding the taxonomy hierarchy.

Skill Taxonomy & Naming

5-level hierarchy (Industry > Sub-Industry > Business Function > Use Case Skill > Sub-Skill). Flat hcls-{sub}-{func}-{skill} naming convention encodes all 5 levels in the directory name.

Orchestrator Loading

Profile uses systemPromptRepo (NOT systemPrompt string) to load the orchestrator agent .md file directly from GitHub. Structure: {ref, source, url}. Ensures orchestrator stays in sync with repo.

CKE Packaging

Standalone composable skills (not embedded in domain skills). Domain skills invoke them on-demand. Preflight checker detects Marketplace listing and guides setup if missing.

Data Model Knowledge

Auto-fires as Step 0 for schema-dependent tasks (DICOM). CKEs are on-demand. Both use preflight checker + setup scripts + graceful fallback pattern.

Sub-Skill Pattern

Router skill (e.g., hcls-provider-imaging) with sub-skills nested inside its directory. Sub-skills declare parent_skill in SKILL.md frontmatter. Router detects intent and dispatches.

Profile Versioning

Incubator: milestone tags (m{n}-{scope}) - lightweight, deletable, for demos. Production: semver (v1.0.0) on SFS repo - immutable, formal releases.

Contribution Flow

Hybrid: field can file issues OR submit PRs on incubator repo. Tiger Team triages all contributions. Only Tiger Team submits promotion PRs to SFS production repo.

Twin Orchestrator

Incubator and production profiles can share routing logic, taxonomy, and guardrails from a single Jinja2 template. Only skill refs and profile metadata differ between the two.

Governance

HIPAA guardrails enforced as cross-cutting concerns across all workflows. Masking, RLS, audit trails, de-identification,

data classification applied by default.

Document generated by Cortex Code | March 2026 | Health Sciences Industry Solutions Tiger Team